Decidability and Undecidability Results for LIA-Definable Impartial Combinatorial Games

Anonymous author
Anonymous affiliation

Abstract

Combinatorial game theory is a branch of mathematics and theoretical computer science that typically studies deterministic games with perfect information and no elements of chance. The majority of combinatorial games are LIA-definable and impartial. In this paper, we provide some decidability and undecidability results of LIA-definable impartial combinatorial games (ICGs). Our theoretical results are:

1. Deciding if an LIA-definable ICG is terminating is undecidable.
2. Deciding if an LIA-definable ICG is cyclic is undecidable.
3. Deciding if a state is winning for an LIA-definable ICG is undecidable.
4. Deciding if an LIA formula exactly characterizes the set of winning states of an LIA-definable ICG is undecidable.
5. Deciding if the set of winning states of an LIA-definable terminating ICG is LIA-definable is undecidable.
6. Decision problems 3 and 4 become decidable for LIA-definable terminating ICGs.

2012 ACM Subject Classification Theory of computation → Computability; Theory of computation → Automata over infinite objects; Theory of computation → Logic and verification

Keywords and phrases Linear Integer Arithmetic, Impartial Combinatorial Game, Counter Machine

Digital Object Identifier 10.4230/LIPIcs.ICALP.2026.1

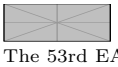
Category Track B: Automata, Logic, Semantics, and Theory of Programming

1 Introduction

Combinatorial game theory is a branch of mathematics and theoretical computer science that typically studies deterministic games with perfect information and no elements of chance. A game that satisfies the following conditions is called a combinatorial game [**?**]: (1) There are two players and (possibly infinitely) many states. (2) Two players alternate moving, each transiting from one state to another. (3) The game ends when it moves to a *terminal state* in which no player has a possible move. Under the *normal play rule*, the last player to move *wins*. Under the misère play rule the last player to move *loses*. If both players has the same choice of moving, then the game is called *impartial*; otherwise, it is called *partizan*. If the number of states is finite, then the game is called *finite*; otherwise, it is called *infinite*. If a game always ends in a finite number of moves, then the game is *terminating*. If a game contains a loop, that is, a sequence of states with repetitions, then the game is *cyclic*.

Researches in combinatorial game theory can be categorized into the three directions.

(1) The theoretical properties of the set of winning (or losing) states and the Sprague-Grundy value of combinatorial games are analyzed. [**?**] analyzed the sufficient and necessary condition of losing states in three-rowed Chomp game when the number of piles in the third row is less than 3. [**?**] showed that some extensions and restrictions of Wythoff's game have exactly the same set of winning states as the original game. [**?**] showed that for any subtraction game with the a finite subtraction set S s.t. the greatest common divisor of S is 1, the Sprague-Grundy sequence is periodic iff $1 \in S$ and S contains only odd integers. Moreover, in the above subtraction game, a state is winning iff the number of remaining


© Anonymous author(s);
licensed under Creative Commons License CC-BY 4.0



chips is odd. [**?**] gave the offset and the period of the Sprague-Grundy sequence of some subtraction games with S where the size of S is greater than 2. [**?**] studied the expansion of subtraction sets in subtraction game without changing the Sprague-Grundy value. [**?**] studied the periodicity of losing states of subtraction games with more than one dimension and two moves.

(2) The decidability and complexity results of some decision problems of combinatorial games are given. One of the most significant decision problems of combinatorial game theory, called *the winning state problem*, is to determine whether a state is winning. The winning state problem in some combinatorial games played on a finite directed graph is PSPACE-complete [**?**, **?**] and that in some popular board games (such as Chess, Checkers, Go, Gobang and Hex) is PSPACE-hard [**?**, **?**, **?**, **?**, **?**]. [**?**] showed that solving the winning state problem in some combinatorial games (including pebble-game, cat and mouse-game, and peg-game) requires exponential time but polynomial time when some parameters are fixed constants. The following two decision problems are undecidable: (1) deciding whether two combinatorial games have the same set of winning states [**?**]; and (2) deciding whether the set of winning states for a combinatorial game is 2-dimensional semi-periodic [**?**].

(3) Some automatic tools are developed for computing the Sprague-Grundy value and the winning strategy in combinatorial games. [**?**] put forward a game-agnostic method for computing Sprague-Grundy value with effective pruning search trees of finite impartial combinatorial games (ICGs). Linear temporal logic (LTL) games are represented by automatas, with objectives specified by LTL formulas. The goal of LTL synthesis is to generate a strategy so as to achieve the objective formula. ICGs can be viewed as a special class of LTL games with reachability objectives, so the methods for LTL synthesis can be directly applied to ICGs. The propositional LTL synthesis problem has been addressed in numerous literature, such as [**?**, **?**, **?**, **?**]. Some works [**?**, **?**, **?**, **?**] solve LTL synthesis problem on linear arithmetic. However, their methods are only able to synthesizes the winning strategy for some specified states, and cannot generalize the strategy to hold for all states. For infinite ICGs defined in linear integer arithmetic, three synthesis methods have been proposed in [**?**, **?**, **?**] to compute the winning formula and the winning strategy for all winning states.

The majority of combinatorial games are impartial and LIA-definable, for example, Subtraction [**?**], Nim [**?**], Wythoff [**?**], Welter [**?**] and Chomp [**?**]. In this paper, we provide some decidability and undecidability results of LIA-definable ICGs under the normal play rule. Our results fall into the second category of researches in combinatorial game theory. The decision problems we study in this paper include:

1. The terminating problem (deciding if an LIA-definable ICG is terminating);
2. The cyclic problem (deciding if an LIA-definable ICG is cyclic);
3. The winning state problem (deciding if a state is winning for an LIA-definable ICG);
4. The winning formula problem (deciding if an LIA formula exactly characterizes the set of winning states of an LIA-definable ICG);
5. The LIA-definability problem of winning formulas (deciding if there is an LIA formula that exactly characterizes the set of winning states of an LIA-definable ICG).

Our theoretical contributions are

1. For general LIA-definable ICGs, the above 5 decision problems are undecidable (Theorems 6, 7, 9, 10 & 13).
2. For LIA-definable terminating ICGs, the winning state and formula problems become decidable (Theorems 16 & 19) but the LIA-definability problem of winning formulas is still undecidable (Theorem 13).

3. The above results also hold for losing and draw states and formulas or under the misère play rule. Due to space limitation, we defer the proofs in Appendix C and D.

We hereafter briefly illustrate the main idea of the proofs in this paper.

- To obtain the undecidability result of the terminating and cyclic problems for LIA-definable ICGs, we provide a reduction from 2-counter machines M to ICGs $\Pi(M)$. The ICG $\Pi(M)$ contains only one single variable v. We encode a configuration C as an integer $2^{C(r_0)}3^{C(r_1)}5^{C(r_2)}b$, where b is not a multiple of 2, 3, or 5, and $C(r_0)$, $C(r_1)$ and $C(r_2)$ denotes the value of the index register r_0 and two counter registers r_1 and r_2 of C, respectively. The moves defined in $\Pi(M)$ emulates the behavior of M. Using this construction, we reduce the mortality and periodic problems of 2-counter machines to the winning state and formula problems for LIA-definable ICGs, respectively. The former two decision problems are undecidable. So are the latter two decision problems.
- To prove the undecidiabiltyundecidability of the winning state and formula problems for LIA-definable ICGs, we prove another reduction $\Pi'(M)$. Similarly to $\Pi(M)$, the $\Pi'(M)$ has only one variable v, but encodes a configuration C as either $3^{C(r_0)}5^{C(r_1)}7^{C(r_2)}b$ or $2^1 3^{C(r_0)}5^{C(r_1)}7^{C(r_2)}b$, where b is not a multiple of 2, 3, 5, or 7. We reduce the halting and mortality problems of 2-counter machines to the winning state and formula problems for LIA-definable ICGs, respectively. Since the former two decision problems are undecidable, the latter two decision problems are undecidable as well.
- To establish the undecidiabilty of the LIA-definability problem of winning formulas for LIA-definable ICGs, we prove the third reduction $\Pi^*(M, C_0)$ where M is a 2-counter machine and C_0 is an initial configuration. The reduction $\Pi^*(M, C_0)$ differs significantly from $\Pi(M)$ and $\Pi'(M)$. The $\Pi^*(M, C_0)$ contains two variables v_1 and v_2, which together defines an infinite grid. We use the cells on a row to encode a configuration C via unary encoding and emulates the run of M on C_0 based on Boolean functions implemented by the moves in $\Pi^*(M, C_0)$. The set of winning states $\Pi^*(M, C_0)$ is LIA-definable iff M halts on C_0. Consequently, the LIA-definability problem of winning formula is undecidable. Moreover, the above undecidability result also hold for LIA-definable terminating ICGs since $\Pi^*(M, C_0)$ is terminating.
- Any terminating ICG has no draw state and all plays are finite. Based on these two properties, we can devise sound and complete algorithm that determines if a state is winning. So the winning state problem for LIA-definable terminating ICGs is decidable. Based on the first property, we construct three LIA-definable constraints, which allow us to reduce the problem of verifying if a formula is the winning formula to the validity problem of LIA, which is decidable. The winning formula problem for LIA-definable terminating ICGs is decidable.

2 Preliminaries

2.1 Boolean Functions

An n-ary Boolean function f is mapping $\{0,1\}^n \to \{0,1\}$. A *literal* is a Boolean variable p or its complement $\bar{p}$. A *minterm* is a conjunction of literals where each variable appears exactly once. For n Boolean variables, there are 2^n distinct minterms. We define the *index* of a minterm τ as $\left[\sum_{i=1}^{n} 2^{i-1} v(p_i)\right] + 1$ where $v(p_i) = 1$ if $p_i \in \tau$; and $v(p_i) = 0$ if $\bar{p}_i \in \tau$. A formula is in *canonical disjunctive normal form (CDNF)*, if it is a disjunction of minterms. It is well-known that every Boolean function f can be expressed in CDNF. We use F_f to denote the set of indices of minterms appearing in the CDNF representation for f.

2.2 Linear Integer Arithmetic

Let $\mathcal{Z}$ be the set of integers and $\mathcal{V}$ the set of variables. *Linear integer arithmetic expressions* (`Exp`), *atoms* (`Atom`) and *formulas* (`Form`) are defined by the following grammar:

$$e, e_1, e_2 \in \texttt{Exp} :: c \mid v \mid e_1 + e_2 \mid e_1 - e_2$$
$$l \in \texttt{Atom} :: e_1 = e_2 \mid e_1 \neq e_2 \mid e_1 < e_2 \mid e_1 > e_2 \mid e_1 \leq e_2 \mid e_1 \geq e_2 \mid e \equiv_{c_1} c_2 \mid e \not\equiv_{c_1} c_2$$
$$\phi, \phi_1, \phi_2 \in \texttt{Form} :: \bot \mid \top \mid l \mid \neg\phi \mid \phi_1 \wedge \phi_2 \mid \phi_1 \vee \phi_2 \mid \forall v\phi \mid \exists v\phi$$

where $c, c_1, c_2 \in \mathcal{Z}$ and $v \in \mathcal{V}$.

The atom $e \equiv_{c_1} c_2$ means that e and c_2 are congruence modulo c_1 (that is, $e - c_2$ is divisible by c_1), and $e \not\equiv_{c_1} c_2$ denotes the negation of $e \equiv_{c_1} c_2$.

A *state* s is an interpretation that maps every variable v to an integer $v(s)$. For a state s, we can evaluate a expression e into an integer $e(s)$ that the expression simplifies to when replacing each variable v with its corresponding value $v(s)$. The Boolean value $\phi(s)$ of a formula ϕ can be defined in a similar way. A state s *satisfies* a formula ϕ, denoted by $s \models \phi$, iff $\phi(s) = \top$. A formula ϕ *implies* another one ψ, denoted by $\phi \models \psi$, iff $s \models \psi$ for every state s satisfying ϕ. A formula ϕ is *valid*, iff $s \models \phi$ for every state s; otherwise, it is *invalid*.

It is well-known that LIA allows for quantifier elimination, that is, any LIA formula can be transformed into an equivalent quantifier-free formula [**?**, **?**]. Let $\mathcal{U}$ be a subset $\{u_1, \cdots, u_n\}$ of variables. We use $\forall\mathcal{U}\phi$ for $\forall u_1 \cdots \forall u_n \phi$ and $\exists\mathcal{U}\phi$ for $\exists u_1 \cdots \exists u_n \phi$.

A set S of integers is *LIA-definable*, iff there is an LIA formula $\phi(x)$ s.t. for every integer z, we have $z \in S$ iff $\phi(z)$ is valid. A fundamental connection between LIA-definable sets and periodicity of the set was established in [**?**] for one-dimensional case. A set S of integers is *ultimately periodic*, iff (1) there are two integers $n_l < 0$ and $p_l > 0$ s.t. for every $x \leq n_l$, $x \in S \cap \{z \mid z \leq n_l\}$ iff $x - p_l \in S \cap \{z \mid z \leq n_l\}$, and (2) there are two integers $n_u > 0$ and $p_u > 0$ s.t. for every $x \geq n_u$, $x \in S \cap \{z \mid z \geq n_u\}$ iff $x + p_u \in S \cap \{z \mid z \geq n_u\}$. A set S of integers is LIA-definable iff S is ultimately periodic[1].

2.3 LIA-Definable Impartial Combinatorial Games

▶ **Definition 1.** An LIA-definable ICG is defined as a tuple $\Pi = \langle \mathcal{V}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ where

- $\mathcal{V}$: a finite set of variables.
- $\mathcal{A}$: a finite set of actions.
- $\mathcal{C}$: an LIA formula denoting all legal states.
- $\mathcal{E}$: an LIA formula denoting all terminal states.

Each *action* transits predecessor states to a possibly infinite set of successor states. To distinguish the predecessor variable and the successor one, we use the unprimed variable $v \in \mathcal{V}$ for the former and the primed one $v' \in \mathcal{V}'$ for the latter. Each action a is defined by a transition formula $\mathcal{T}(a)$ over primed and unprimed variables. For example, the transition formula $v \geq 3 \wedge v' = v - 3$ means that the action is executable when $v \geq 3$ and decreases the value of v by 3. A state s' is a *successor state* of s by performing action a, iff $s \cup s' \models \mathcal{T}(a)$ where s is an interpretation on $\mathcal{V}$ and s' is on $\mathcal{V}'$. *The global transition formula* $\mathcal{T}(\mathcal{A})$ of Π is defined as $\bigvee_{a \in \mathcal{A}} \mathcal{T}(a)$. It represents the whole state transition system of the game Π.

Any state can be classified as three categories: *winning*, *losing* and *draw*.

[1] The original theorem in [**?**] is stated over natural numbers (non-negative integers). However, it can be extended to integers (including negative integers).

▶ **Definition 2.** In an ICG, the sets $\mathcal{W}$ of winning states, $\mathcal{L}$ of losing states and $\mathcal{D}$ of draw states are recursively defined as

1. Every terminal state is a losing state.
2. Every state such that some of its successor states are losing states is a winning state.
3. Every state whose successor states are winning states is a losing state.
4. Every state such that none of its successor states is a losing state and some of its successor states are draw states are draw states.

The three sets $\mathcal{W}$, $\mathcal{L}$ and $\mathcal{D}$ are pairwise disjoint. The above definition characterizes the concepts of $\mathcal{W}$, $\mathcal{L}$ and $\mathcal{D}$ under the normal play rule. Under the misère rule, the definitions of $\mathcal{W}$, $\mathcal{L}$ and $\mathcal{D}$ are the same as Definition 2 except that every terminal state is a winning state. For the sake of clarity, throughout this paper, we assume that ICGs follow the normal play rule. The theoretical results in this paper can be extended to the misère play rule with minor adjustments to the proofs. Due to space limitation, we defer these adjustments in Appendix.

To obtain the finite representation of winning (resp. losing/draw) states, we use an arithmetic formula to exactly capture this notion, called *the winning (resp. losing/draw) formula.*

▶ **Definition 3.** Let $\Pi = \langle \mathcal{V}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be a terminating ICG. We say a formula $\phi \in \texttt{Form}$ is a *winning (resp. losing/draw) formula* of Π, iff for every state s, $s \models \mathcal{C} \wedge \phi$ iff s is a legal winning (resp. losing/draw) state.

The winning formula ϕ along with the formula $\mathcal{C}$ exactly characterizes the set of winning states. In general, there are many logical different winning formulas for an ICG. But they are logically equivalent under the background theory $\mathcal{C}$, that is, $\mathcal{C} \models \phi \equiv \phi'$. Hence, we sometimes call a winning formula the winning one. The concepts of losing and draw formulas are similar.

2.4 2-Counter Machine

▶ **Definition 4** ([?]). A *2-counter machine* M consists of

1. a special index register r_0 which denotes the label of current instruction to be executed;
2. two count registers r_1 and r_2, each of which can hold a non-negative integer;
3. a list L of labeled instructions, which comprises seven types: $\texttt{Halt}$, $\texttt{Inc}_1$, $\texttt{Inc}_2$, $\texttt{Dec}_1$, $\texttt{Dec}_2$, $\texttt{JZ}_{1,j}$ and $\texttt{JZ}_{2,j}$ where $1 \leq j \leq |L|$ and $|L|$ denotes the length of L.

A *configuration* C of a counter machine M is a mapping that associate every register r to a non-negative integer $C(r)$. A configuration C is a *halting* configuration, iff $C(r_0) : \texttt{Halt} \in L$.

Intuitively, $\texttt{Inc}_i$ increments the value of the i-th count register r_i and proceeds to the next instruction on L, while leaving all other count registers unchanged. The instruction $\texttt{Dec}_i$ behaves like $\texttt{Inc}_i$ except it decrements the value of r_i when $C(r_i) > 0$ and maintains the value of r_i when $C(r_i) = 0$. The instruction $\texttt{JZ}_{i,l}$ tests whether the value of the i-th count register equals 0. If so, it jumps to the l-th instruction; otherwise, it moves to the next one.

We use $M(C)$ to denote the single step configuration of M on C, obtained from C via executing the current instruction. We use $M^i(C)$ to denote the i-th step configuration of M on C, which is defined as $M^1(C) = M(C)$ and $M(M^{i-1}(C))$ for $2 \leq i \leq n$. A 2-counter machine M is *terminating* in a configuration C, iff there is a run $C, M^1(C), \cdots, M^m(C)$ s.t. $M^m(C)$ is a halting configuration. The *halting problem* is to decide if M is terminating in a configuration C and the *mortality problem* is to decide if M is terminating on all configurations. It is well-known that the halting problem of 2-counter machines is undecidable [?]. The undecidability result still hold for the mortality problem [?]. A configuration C is *periodic* iff $C = M^m(C)$ for $m \geq 0$. The *periodic problem* is to decide if M has a periodic configuration. The periodic problem of 2-counter machines is undecidable [?].

3 Undecidabilty Results

In this section, we prove that the decision problems illustrated in Introduction for LIA-definable ICGs are undecidable.

3.1 The Terminating and Cyclic Problems

To prove the undecidability of the terminating and cyclic problems, we first provide a reduction from a 2-counter machine M to an ICG $\Pi(M)$.

Given three positive integers p, m and n s.t. p is a prime number and $m \leq n$, we define an LIA-formula $\varphi_{p,m,n}$ as $\bigvee_{1\leq b\leq p^{n+1}}(b \not\equiv_p 0 \wedge v \equiv_{p^{n+1}} b \cdot p^m)$ where b ranges over integers and v is a variable. It means that the value of v is $c \cdot p^m$ where c is not a multiple of p.

▶ **Definition 5.** Let M be a 2-counter machine and L the list of labeled instructions of M. An ICG $\Pi(M) = \langle \mathcal{V}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ is defined as

- $\mathcal{V} : \{v\}$ and $\mathcal{A} : \{a_l \mid 1 \leq l \leq |L|\}$;
- $\mathcal{C} : v \geq 0 \wedge \left(\bigvee_{1\leq l\leq |L|} \varphi_{2,l,|L|}\right)$ and $\mathcal{E} : \bigvee_{l:\texttt{Halt}\in L} \varphi_{2,l,|L|}(v)$;

The transition formula $\mathcal{T}(a_l)$ for action a_l is defined as

$$\mathcal{T}(a_l) = \begin{cases} \varphi_{2,l,|L|} \wedge v' = 6v, & \text{if } l : \texttt{Inc}_1 \in L; \\ \varphi_{2,l,|L|} \wedge v' = 10v, & \text{if } l : \texttt{Inc}_2 \in L; \\ \varphi_{2,l,|L|} \wedge \left[(v \equiv_3 0 \wedge 3v' = 2v) \vee (v \not\equiv_3 0 \wedge v' = 2v)\right], & \text{if } l : \texttt{Dec}_1 \in L; \\ \varphi_{2,l,|L|} \wedge \left[(v \equiv_5 0 \wedge 5v' = 2v) \vee (v \not\equiv_5 0 \wedge v' = 2v)\right], & \text{if } l : \texttt{Dec}_2 \in L; \\ \varphi_{2,l,|L|} \wedge \left[(v \equiv_3 0 \wedge 2^l v' = 2^j v) \vee (v \not\equiv_3 0 \wedge v' = 2v)\right], & \text{if } l : \texttt{JZ}_{1,j} \in L; \\ \varphi_{2,l,|L|} \wedge \left[(v \equiv_5 0 \wedge 2^l v' = 2^j v) \vee (v \not\equiv_5 0 \wedge v' = 2v)\right], & \text{if } l : \texttt{JZ}_{2,j} \in L. \end{cases}$$

The ICG $\Pi(M)$ uses a single variable v to encode the index register and the two count registers of the 2-counter machine M. A positive integer z denotes a configuration C, iff $z = 2^{C(r_0)}3^{C(r_1)}5^{C(r_2)}b$, where b is not a multiple of 2, 3, or 5. The constraint $\mathcal{C}$ ensures that the value of index register is in the valid range (*i.e.*, $1 \leq C(r_0) \leq |L|$) in every legal state. The termination condition $\mathcal{E}$ means that the execution of instructions ends in instruction `Halt`. The executability condition of action a_l is $\varphi_{2,l,|L|}$, indicating that it corresponds to the l-th instruction. Suppose that $l : \texttt{Inc}_1 \in L$ and s is a state with the value $v(s) = 2^{C(r_0)}3^{C(r_1)}5^{C(r_2)}b$. Action a_l multiples the value of v by 6 (*i.e.*, $v(s') = 2^{C(r_0)+1}3^{C(r_1)+1}5^{C(r_2)}b$). In fact, it increments the values of r_0 and r_1, thereby emulating instruction $\texttt{Inc}_1$. Similarly, action a_l emulates its corresponding instruction for the other cases. Since the three symbols b, l and j denote fixed integers in the constraint $\mathcal{C}$, the termination condition $\mathcal{E}$ and transition formulas $\mathcal{T}(a)$ are LIA-definable. So is $\Pi(M)$.

▶ **Theorem 6.** *The terminating problem for LIA-definable ICGs is undecidable.*

Proof. It can be verified that a 2-counter machine M halts on all configurations iff the ICG $\Pi(M)$ starting from any state eventually ends. Since the mortality problem of 2-counter machines is undecidable, deciding if an LIA-definable ICG is terminating is undecidable. ◀

▶ **Theorem 7.** *The cyclic problem for LIA-definable ICGs is undecidable.*

Proof. It can be verified that a 2-counter machine M has a periodic configuration iff $\Pi(M)$ contains a loop. Since the periodic problem of 2-counter machine is undecidable, deciding if an LIA-definable ICG is cyclic is also undecidable. ◀

3.2 The Winning State and Formula Problems

We now show that the winning state and formula problems are undecidable. To this end, we provide another reduction from 2-counter machines to ICGs.

▶ **Definition 8.** Let M be a 2-counter machine and L the list of labeled instructions of M. An ICG $\Pi'(M) = \langle \mathcal{V}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ is defined as

- $\mathcal{V} : \{v\}$ and $\mathcal{A} : \{a'_l \mid 1 \leq l \leq |L| + 1\}$;
- $\mathcal{C} : v \geq 0 \wedge v \not\equiv_4 0 \wedge \left(\bigvee_{1 \leq l \leq |L|} \varphi_{3,l,|L|}\right)$ and $\mathcal{E} : v \not\equiv_4 0 \wedge v \not\equiv_4 2 \wedge \left(\bigvee_{l:\texttt{Halt} \in L} \varphi_{3,l,|L|}\right)$;

The transition formula $\mathcal{T}(a'_l)$ for action a'_l is defined as

$$\mathcal{T}(a'_l) = \begin{cases} \varphi_{3,l,|L|} \wedge v \equiv_4 2 \wedge (2v' = 15v), & \text{if } l : \texttt{Inc}_1 \in L; \\ \varphi_{3,l,|L|} \wedge v \equiv_4 2 \wedge (2v' = 21v), & \text{if } l : \texttt{Inc}_2 \in L; \\ \varphi_{3,l,|L|} \wedge v \equiv_4 2 \wedge [(v \equiv_5 0 \wedge 10v' = 3v) \vee (v \not\equiv_5 0 \wedge 2v' = 3v)], & \text{if } l : \texttt{Dec}_1 \in L; \\ \varphi_{3,l,|L|} \wedge v \equiv_4 2 \wedge [(v \equiv_7 0 \wedge 14v' = 3v) \vee (v \not\equiv_7 0 \wedge 2v' = 3v)], & \text{if } l : \texttt{Dec}_2 \in L; \\ \varphi_{3,l,|L|} \wedge v \equiv_4 2 \wedge \left[(v \equiv_5 0 \wedge 2 \cdot 3^l v' = 3^j \cdot v) \vee (v \not\equiv_5 0 \wedge 2v' = 3v)\right], & \text{if } l : \texttt{JZ}_{1,j} \in L; \\ \varphi_{3,l,|L|} \wedge v \equiv_4 2 \wedge \left[(v \equiv_7 0 \wedge 2 \cdot 3^l v' = 3^j \cdot v) \vee (v \not\equiv_7 0 \wedge 2v' = 3v)\right], & \text{if } l : \texttt{JZ}_{2,j} \in L; \\ \varphi_{3,l,|L|} \wedge v \not\equiv_4 0 \wedge v \not\equiv_4 2 \wedge v' = 2v, & \text{if } l = |L| + 1. \end{cases}$$

Definition 8 is a slight modification of Definition 5. In Definition 8, a positive integer z denotes a configuration C, iff $z = 3^{C(r_0)}5^{C(r_1)}7^{C(r_2)}b$ or $z = 2^1 3^{C(r_0)}5^{C(r_1)}7^{C(r_2)}b$, where b is not a multiple of 2, 3, 5, or 7. The constraint $\mathcal{C}$ requires the value $v(s)$ of every legal state s to be of the form $3^l 5^m 7^n b$ or $2^1 3^l 5^m 7^n b$ where $1 \leq l \leq |L|$. The termination condition $\mathcal{E}$ stipulates that the value $v(s)$ of every terminal state s is of the form $3^l 5^m 7^n b$ where $l : \texttt{Halt} \in L$. In addition, for $1 \leq l \leq |L|$, each action a'_l in the ICG $\Pi'(M)$ transits an integer $2^1 3^{C(r_0)}5^{C(r_1)}7^{C(r_2)}b$ to another one $3^{M(C)(r_0)}5^{M(C)(r_1)}7^{M(C)(r_2)}b$, which emulates the l-th instruction of L. The last action $a'_{|L|+1}$ multiples the value of v by 2, which essentially transiting an integer $3^{C(r_0)}5^{C(r_1)}7^{C(r_2)}b$ to $2^1 3^{C(r_0)}5^{C(r_1)}7^{C(r_2)}b$. It can be verified that an integer of the form $2^1 3^l 5^m 7^n b$ (resp. $3^l 5^m 7^n b$) corresponds to a winning (resp. losing) state. Each run of $\Pi'(M)$ alternate between winning states and losing states.

▶ **Theorem 9.** *The winning state problem for LIA-definable ICGs is undecidable.*

Proof. Let s be a state of $\Pi'(M)$ and C the corresponding configuration in M s.t. $v(s) = 2^1 3^{C(r_0)}5^{C(r_1)}7^{C(r_2)}b$. The state s is winning iff M is terminating in C. Since the halting problem of 2-counter machines is undecidable, deciding if a state is winning for an LIA-definable ICG is undecidable. ◀

▶ **Theorem 10.** *The winning formula problem for LIA-definable ICGs is undecidable.*

Proof. Let ϕ be the formula $v \geq 0 \wedge v \equiv_4 2 \wedge \left(\bigvee_{1 \leq l \leq |L|} \varphi_{3,l,|L|}\right)$. The formula ϕ exactly characterizes the set of positive integers $2^1 3^l b$ where $1 \leq l \leq |L|$ and b is not a multiple of 2 or 3. It can be verified that ϕ is the winning formula for $\Pi'(M)$ iff M is terminating on all configurations. Since the mortality problem of 2-counter machines is undecidable, deciding if an LIA-formula is a winning formula for an LIA-definable ICG is undecidable. ◀

3.3 The LIA-Definability Problem of Winning Formulas

[?] proved that the set of winning states of any Subtraction game is LIA-definable. However, not all LIA-definable ICG has an LIA-definable set of winning states. [?] introduced an LIA-definable ICG, namely Mark game, and showed that its set of winning states is not ultimately

periodic. Mark game is a variant of Subtraction game. In this game, the player can take either move chips to 1 or $\lfloor \frac{v}{2} \rfloor$ chips from the table, where v denotes the number of remaining chips. The transition formula of action *take* in Mark game is $v \geq 0 \wedge (v' = v-1 \vee 2v' = v \vee 2v' = v-1)$.

We close this section by proving the undecidability of the LIA-definability problem of winning formulas for LIA-definable ICGs. We provide the third reduction from 2-counter machines M and their initial configurations C_0 to LIA-definable ICGs $\Pi^*(M, C_0)$. The ICG $\Pi^*(M, C_0)$ not only emulates the run of M on C_0 but also embeds Mark game.

The ICG $\Pi^*(M, C_0)$ involves two variables v_1 and v_2, which together define an infinite grid. Every state s corresponds to a cell (x, y) on column x and row y where $x = v_1(s)$ and $y = v_2(s)$. We introduce a Boolean variable $w(x, y)$ to indicate if state s is winning. If s is winning, then $w(x, y) = 1$; otherwise, $w(x, y) = 0$.

We encode each configuration C into row y using unary encoding. We use cells $(1, y), (2, y), \cdots, (|L|, y)$ to represent the value $C(r_0)$ of r_0 where $C(r_0)$ is the unary number $w(1, y)w(2, y) \cdots w(|L|, y)$ and $w(1, y)$ is the least significant digit. In the same way, we use cells $(|L| + 1, y), (|L| + 3, y) \cdots$ to denote the value $C(r_1)$ and $(|L| + 2, y), (|L| + 4, y) \cdots$ for $C(r_2)$. If C is not the initial configuration C_0, we use $w(0, y)$ to denote if C is halting. If C is halting, then $w(0, y) = 01$; otherwise, $w(0, y) = 10$.

Suppose that row y_i corresponds to the i-th step configuration $M^i(C_0)$ and y_{i-1} to $M^{i-1}(C_0)$. The Boolean value $w(x, y_i)$ for cell (x, y_i) depends on those of some cells on row y_{i-1}. For $1 \leq x \leq |L|$, the Boolean value of $w(x, y_i)$ depends on the value of index register r_0 on the previous configuration $M^{i-1}(C_0)$, as well as the two Boolean values of $w(|L|+1, y_{i-1})$ and $w(|L|+2, y_{i-1})$ that determine if $M^{i-1}(C_0)(r_1)$ and $M^{i-1}(C_0)(r_2)$ are zero, respectively. Moreover, since $M^{i-1}(C_0)(r_0) = w(1, y_{i-1}) \cdots w(|L|, y_{i-1})$, $w(x, y_i)$ can be expressed as a Boolean function f_x over $w(1, y_{i-1}), \cdots, w(|L| + 2, y_{i-1})$. For $x = |L| + 1$, $w(x, y_i)$ can be defined by a Boolean function $f_{|L|+1}$ over $w(1, y_{i-1}), \cdots, w(|L|, y_{i-1}), w(|L| + 1, y_{i-1})$ and $w(|L| + 3, y_{i-1})$. For $x = |L| + 3, |L| + 5, \cdots$, the value of each $w(x, y_i)$ is determined by a Boolean function $f_{|L|+3}$ over $w(1, y_{i-1}), \cdots, w(|L|, y_{i-1}), w(x - 2, y_{i-1}), w(x, y_{i-1})$ and $w(x + 2, y_{i-1})$. All Boolean variables $w(y_i, x)$ s.t. $x \in \{|L| + 3, |L| + 5, \cdots\}$ shares the same function $f_{|L|+3}$ but differ in their input variables. Similarly, $w(0, y_i)$ and $w(|L| + 2, y_i)$ corresponds to Boolean functions f_0 and $f_{|L|+2}$, respectively. For $x = |L| + 4, |L| + 6, \cdots$, each Boolean variable $w(x, y_i)$ corresponds to a Boolean function $f_{|L|+4}$. We use totally $|L| + 5$ Boolean functions to emulate the single-step update of configurations over unary encoding. Due to space limitation, we present the definitions of functions f_x's in Appendix.

Let $n = |L| + 3$ and $m = 2^n + n + 2$. For technical convenience, we make the following assumptions: (1) each function f_x has n input variables; (2) the last instruction of L is `Halt`; and (3) the next configuration of a halting configuration is itself. If a function has less than n input variables, we can add some independent variables without affecting its Boolean value. We use m rows to emulate the above Boolean functions f_x's. The initial configuration C_0 is encoded into row 1 while the i-th step configuration $M^i(C_0)$ into row $i \cdot m + 1$.

Let (v_1, v_2) be a cell on a row that encodes the i-th step configuration $M^i(C_0)$ and f_j the Boolean function for (v_1, v_2) where v_1 is the column index and $v_2 = i \cdot m + 1$ is the row index. To determine the k-th input variable of f_j for $w(v_1, v_2)$, we define a mapping $o(j, k)$ from a pair of integers (j, k) to an expression. Intuitively, $o(j, k)$ denotes the offset of the column index x of cell (x, v_2') relative to the column index v_1 of cell (v_1, v_2) where $v_2' = (i - 1) \cdot m + 1$ and $w(x, v_2')$ is the k-th input variable of f_j for $w(v_1, v_2)$. Hence, $w(v_1, v_2) = f_j(w(v_1 + o(j, 1), v_2'), w(v_1 + o(j, 2), v_2'), \cdots, w(v_1 + o(j, n), v_2'))$.

$$o(j,k) = \begin{cases} k-j, & \text{if } 0 \le j \le |L|+2; \\ k-v_1, & \text{if } j = |L|+3, |L|+4 \text{ and } 1 \le k \le |L|; \\ -2, & \text{if } j = |L|+3, |L|+4 \text{ and } k = |L|+1; \\ 0, & \text{if } j = |L|+3, |L|+4 \text{ and } k = |L|+2; \\ 2, & \text{if } j = |L|+3, |L|+4 \text{ and } k = |L|+3. \end{cases}$$

For $1 \le k \le n$, cell $(v_1, v_2'+k)$ corresponds to the complement of the k-th input variable $w(v_1+o(j,k), v_2')$ (that is, $w(v_1, v_2'+k) = \neg w(v_1+o(j,k), v_2')$). This is because there is a unique successor state $(v_1+o(j,k), v_2')$ of cell $(v_1, v_2'+k)$ due to the effect of $neglit_{j,k}$. Similarly, actions $term_{j,k,t}$'s stipulate that cell $(v_1, v_2'+n+t)$ corresponds to the t-th minterm τ_t over variables $w(v_1+o(j,1), v_2'), \cdots, w(v_1+o(j,n), v_2')$ (that is, $w(v_1, v_2'+n+t) = \tau_t$). Actions $ndnf_{j,t}$'s ensure that cell $(v_1, v_2'+m)$ corresponds to the negation of f_j over variables $w(v_1+o(j,1), v_2'), \cdots, w(v_1+o(j,n), v_2')$ (that is, $w(v_1, v_2'+m) = \bigwedge_{t \in F_{f_j}} \neg\tau_t$) where F_{f_j} is the set of indices of minterms appearing in the CDNF formula for f_j. Action dnf ensures that $w(v_1, v_2)$ has the same Boolean value as f_j (that is, $w(v_1, v_2) = f_j$).

We embed Mark game into $\Pi^*(M, C_0)$ using two actions *take* and *connect*. Action *take* decreases the value of v_1 by 1 or $\lfloor \frac{v_1}{2} \rfloor$ on row 0 while action *connect* links all alldel all cells with row 0 to all cells $(0, i \cdot m+1)$ for $i \ge 1$. It can be verified that (1) if M is not terminating in C_0, then $w(x,0) = 1$ iff x is a winning state in Mark game; and (2) if M is terminating in C_0, then thendel then $w(x,0) = 1$.

Finally, since the initial configuration C_0, with $C_0(r_0) = 1$, and $C_0(r_1) = C_0(r_2) = 0$, is encoded in row 1, the terminal condition is defined as $\mathcal{E} = v_1 \ne 1 \wedge v_2 = 1$. We also let the non-terminal state $(1,1)$ connect the terminal state $(0,1)$ via action *init*. Notice that the state $(0,1)$ is always a terminal state whether or not C_0 is halting.

The definition of $\Pi^*(M, C_0)$ is in the following.

▶ **Definition 11.** Let M be a 2-counter machine, C_0 the initial configuration and L the list of labeled instructions of M. An ICG $\Pi^*(M, C_0) = \langle \mathcal{V}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ is defined as

- $\mathcal{V} : \{v_1, v_2\}$;
- $\mathcal{A} : \{init, take, connect, dnf\} \cup \{ndnf_{j,t} \mid 1 \le j \le |4L|+1, t \in F_{f_j}\} \cup$
  $\{neglit_{j,k}, term_{j,k,t} \mid 0 \le j \le |L|+4, 1 \le k \le n, 1 \le t \le 2^n\}$;
  where $n = |L|+3$, $m = 2^n + n + 2$;
- $\mathcal{C} : v_1 \ge 0 \wedge v_2 \ge 0$;
- $\mathcal{E}$: $\left[\bigvee_{1 \le i \le C_0(r_0)} v_1 \ne i \vee \bigvee_{1 \le i \le C_0(r_1)} (v_1 \ne 2 \cdot i + |L| - 1) \vee \bigvee_{1 \le i \le C_0(r_2)} (v_1 \ne 2 \cdot i + |L|)\right] \wedge v_2 = 1$.

The transition formulas for all actions are defined as

- $\mathcal{T}(init) : \left[\bigvee_{1 \le i \le C_0(r_0)} v_1 \ne i \vee \bigvee_{1 \le i \le C_0(r_1)} (v_1 \ne 2 \cdot i + |L| - 1) \vee \bigvee_{1 \le i \le C_0(r_1 r_2)} (v_1 \ne 2 \cdot i + |L|)\right] \wedge$
  $v_1' = 0 \wedge v_2' = 1$;
- $\mathcal{T}(take) : v_1 > 0 \wedge v_2 = 0 \wedge (v_1' = v_1 - 1 \vee 2v_1' = v_1 \vee 2v_1' = v_1 - 1)$;
- $\mathcal{T}(connect) : v_1 \ge 0 \wedge v_2 = 0 \wedge v_1' = 0 \wedge v_2' \equiv_{ym} 1 \wedge v_2' > 1$;
- $\mathcal{T}(neglit_{j,k}) : \delta(j) \wedge v_2 \equiv_m k+1 \wedge v_1' = v_1 + o(j,k) \wedge v_2' = v_2 - k$;
- $\mathcal{T}(term_{j,k,t}) : \begin{cases} \delta(j) \wedge v_2 \equiv_m n+t+1 \wedge v_1' = v_1 \wedge v_2' = v_2 - n - t + o(j,k), & \text{if } p_k \in \tau_t; \\ \delta(j) \wedge v_2 \equiv_m n+t+1 \wedge v_1' = v_1 + o(j,k) \wedge v_2' = v_2 - n - t, & \text{if } \bar{p}_k \in \tau_t. \end{cases}$
- $\mathcal{T}(ndnf_{j,t}) : \delta(j) \wedge v_2 > 0 \wedge v_2 \equiv_m 0 \wedge v_1' = v_1 \wedge v_2' = v_2 - 2^n + t$;
- $\mathcal{T}(dnf) : v_1 > 0 \wedge v_2 > 1 \wedge v_2 \equiv_m 1 \wedge v_1' = v_1 \wedge v_2' = v_2 - 1$.

where $\delta(j)$ is defined as

$$\delta(j) = \begin{cases} v_1 = j, & \text{if } 0 \le j \le |L| + 2; \\ v_1 \equiv_2 1 \wedge v_1 \ge |L| + 3, & \text{if |L| is even and } j = |L| + 3, |L| + 5, \cdots; \\ v_1 \equiv_2 0 \wedge v_1 \ge |L| + 3, & \text{if |L| is odd and } j = |L| + 3, |L| + 5, \cdots; \\ v_1 \equiv_2 0 \wedge v_1 \ge |L| + 4, & \text{if |L| is even and } j = |L| + 4, |L| + 6, \cdots; \\ v_1 \equiv_2 1 \wedge v_1 \ge |L| + 4, & \text{if |L| is odd and } j = |L| + 4, |L| + 6, \cdots. \end{cases}$$

The following lemma says that all rows $i \cdot m + 1$'s emulate the run of a 2-counter machine M on the initial configuration C_0.

▶ **Lemma 12.** *Let M be a 2-counter machine, C_0 the initial configuration, and $\Pi^*(M, C_0)$ the ICG defined in Definition 11. Let $y_i = i \cdot m + 1$. Then, for $i \ge 1$,*

- $M^i(C_0)(r_0) = w(1, y_i) \cdots w(|L|, y_i)$;
- $M^i(C_0)(r_1) = w(|L| + 1, y_i) w(|L| + 3, y_i) \cdots$
- $M^i(C_0)(r_2) = w(|L| + 2, y_i) w(|L| + 4, y_i) \cdots$
- $M^i(C_0)$ *is halting iff* $w(0, y_i) = 1$.

▶ **Theorem 13.** *The LIA-definability problem of winning formulas for LIA-definable ICGs is undecidable.*

Proof. We prove that M halts in C_0 iff the set of winning states in $\Pi^*(M, C_0)$ is LIA-definable.

Suppose that M halts on C_0. Let h be the integer s.t. $M^h(C_0)$ is the first halting configuration. Since $C_0(r_1) = C_0(r_2) = 0$, there are only finite number of cells (x, y) with $w(x, y) = 1$ for $1 \le y \le h \cdot m + 1$. We can use an LIA formula φ to exactly describe the above cells. In addition, any configuration $M^{h'}(C_0)$ is the same as $M^h(C_0)$ for $h' > h$. So $w(x, y) = w(x, y + m + 1)$ for $y > h \cdot m + 1$. We can use an LIA formula ϕ to exactly characterize the set of cells (x, y) where $y > h \cdot m + 1$ and $w(x, y) = 1$. Finally, all cells on row 0 connects cell $(0, h \cdot m + 1)$ with $w(0, h \cdot m + 1) = 0$. So $w(x, 0) = 1$ for any column x, which can be expressed in the LIA formula $v_1 2 = 0$. Hence, the set of winning states in $\Pi^*(M, C_0)$ is LIA-definable.

Suppose that M does not halt on C_0. Since row 0 emulates Mark game, the set of winning states on row 0 in not LIA-definable. Nor is the set of winning states of $\Pi^*(M, C_0)$.

Since the halting problem of M is undecidable, deciding if the set of winning states of $\Pi^*(M, C_0)$ is LIA-definable is undecidable. In addition, it can be verified that $\Pi^*(M, C_0)$ is terminating. This undecidability result holds for LIA-definable terminating ICGs. ◀

4 Decidability Results for Terminating ICGs

In this section, we provide two decidable results of the winning state and formula problems for terminating ICGs.

Since an terminating ICG starting any state eventually ends at a terminal state, so it has no draw state. Definition 2 can be simplified as the following.

▶ **Definition 14.** In a terminating ICG, the sets $\mathcal{W}$ of winning states and $\mathcal{L}$ of losing states are recursively defined as

1. All terminal states are losing states.
2. All states such that there is at least one move to a losing state are winning states.
3. All states such that the only possible moves are to winning states are losing states.

Based on Definition 14, we design a recursive algorithm, illustrated in Algorithm 1 to check if any legal state is winning. Algorithm 1 proceeds by backward induction. It iteratively

Algorithm 1 `IsWinningState`(Π, s)

Input: $\Pi = \langle \mathcal{V}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$: an LIA-definable ICG;
s: a legal state.
Output: $\top$, if s is a winning state; $\bot$, otherwise.

1 $\psi \leftarrow (\mathcal{C} \wedge \mathcal{E})$ and $\varphi \leftarrow \bot$
2 **if** $s \models \psi$ **then return** $\bot$;
3 **else**
4 **while true do**
5 $\varphi' \leftarrow$ a quantifier-free formula equivalent to $\exists \mathcal{V}' \{\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \psi)[\mathcal{V}/\mathcal{V}']\}$
6 $\varphi \leftarrow \varphi \vee \varphi'$
7 **if** $s \models \mathcal{C} \wedge \varphi$ **then return** $\top$;
8 $\psi' \leftarrow$ a quantifier-free formula equivalent to $\forall \mathcal{V}' \{\mathcal{T}(\mathcal{A}) \rightarrow (\mathcal{C} \wedge \varphi)[\mathcal{V}/\mathcal{V}']\}$
9 $\psi \leftarrow \psi \vee \psi'$
10 **if** $s \models \mathcal{C} \wedge \psi$ **then return** $\bot$;

constructs two formulas: ψ and φ, representing the two increasing sets of losing states and of winning states, respectively. Initially, the formula ψ is $\mathcal{C} \wedge \mathcal{E}$, corresponding to Condition 1 of Definition 14, and φ is $\bot$, meaning that no terminal state is winning (line 1). Then, Algorithm 1 enters a loop (lines 4 - 10). According to Condition 2 of Definition 14, we construct the formula φ' from the current formula ψ, representing the subset of winning states that have at least one successor state satisfying φ (line 5). The notation $(\mathcal{C} \wedge \psi)[\mathcal{V}/\mathcal{V}']$ denotes the formula obtained by replacing every occurrence of $v \in \mathcal{V}$ in $\mathcal{C} \wedge \psi$ with v'. The disjunction of φ' and the original φ is the new subset of winning states (line 6). Similarly, the disjunction of ψ' and the original ψ is the new subset of losing states where ψ' denotes the subset of losing states all of whose successor states satisfying φ (line 8 - 9). If s satisfies ψ, then s is a losing state (lines 2 & 10). Similarly, s is a winning state when $s \models \varphi$ (line 7).

Algorithm 1 enjoys the termination, soundness and completeness properties.

▶ **Lemma 15.** *Let Π be an LIA-definable ICG and s a legal state. Then,*

- *Termination: Algorithm* `IsWinningState`(Π, s) *always terminates.*
- *Soundness: If* `IsWinningState`(Π, s) *returns $\top$, then s is a winning state.*
- *Completeness: If s is a winning state, then* `IsWinningState`(Π, s) *returns $\top$.*

▶ **Theorem 16.** *The winning state problem for LIA-definable terminating ICGs is decidable.*

Proof. It directly follows from Lemma 15 that deciding a state is winning for an LIA-definable terminating ICG is decidable. ◀

We provide three constraints for verifying if a formula ϕ is the winning formula. Each of the above constraints corresponds to each condition of winning and losing states (cf. Definition 14) of terminating ICGs.

▶ **Definition 17.** Let $\Pi = \langle \mathcal{V}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be a terminating ICG. The constraints for the winning formula ϕ of Π are as follows:

1. $\mathcal{C} \wedge \mathcal{E} \rightarrow \neg\phi$;
2. $(\mathcal{C} \wedge \phi) \rightarrow \exists \mathcal{V}' \{\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \neg\phi)[\mathcal{V}/\mathcal{V}']\}$;
3. $(\mathcal{C} \wedge \neg\phi) \rightarrow \forall \mathcal{V}' \{\mathcal{T}(\mathcal{A}) \rightarrow (\mathcal{C} \wedge \phi)[\mathcal{V}/\mathcal{V}']\}$.

The following lemma confirms the correctness of the constraints for the winning formula.

▶ **Lemma 18.** *Let* $\Pi = \langle \mathcal{V}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ *be a terminating ICG. A formula* ϕ *is the winning formula of* Π *iff all of the constraints illustrated in Definition 17 are valid.*

▶ **Theorem 19.** *The winning formula problem for LIA-definable terminating ICGs is decidable.*

Proof. Lemma 18 reduces the winning formula problem for an LIA-definable terminating ICG to the validity problem of LIA. Thanks to the decidability of the latter problem [**?**][2], the former decision problem is decidable. ◀

5 Conclusions

In this paper, we have proven the terminating, cyclic, winning state, winning formula and the LIA-definability of winning formulas problems are all undecidable for LIA-definable ICGs. For LIA-definable terminating ICG, however, the winning state and formula problems become decidable. All of the above results also extend to losing and draw states and formulas as well as to the misère play rule. Two open questions remain for future work. The decidability of the LIA-definability problems of draw formulas for LIA-definable ICGs and of single-variable LIA-definable ICGs are unknown.

A Details of the Third Reduction $\Pi^*(M, C_0)$

We first define each Boolean function f_j based on a list L of labeled instructions. Suppose that each f_j has input variables $p_1, p_2, \cdots, p_{|L|+3}$.

$f_0 = \bigvee_{l:\texttt{Halt} \in L \text{ and } 1 \le l < |L|} (f_l \wedge \neg f_{l+1}) \vee f_{|L|}$;

$f_1 = \top$;

$f_j = \bigvee_{l=1}^{|L|} \varphi_{j,l}$ for $2 \le j \le |L|$ where $\varphi_{j,l}$ is defined as follows:

$$\varphi_{j,l} = \begin{cases} p_l \wedge \bar{p}_{l+1} \wedge p_{j-1}, & \text{if the } l\text{-th instruction of } L \text{ is } \texttt{Inc}_1, \texttt{Inc}_2, \texttt{Dec}_1 \text{ or } \texttt{Dec}_2; \\ p_l \wedge \bar{p}_{l+1} \wedge (\bar{p}_{|L|+1} \vee p_{|L|+1} \wedge p_{j-1}), & \text{if the } l\text{-th instruction of } L \text{ is } \texttt{JZ}_{1,k} \text{ and } k \le j; \\ p_l \wedge \bar{p}_{l+1} \wedge p_{|L|+1} \wedge p_{j-1}, & \text{if the } l\text{-th instruction of } L \text{ is } \texttt{JZ}_{1,k} \text{ and } k > j; \\ p_l \wedge \bar{p}_{l+1} \wedge (\bar{p}_{|L|+2} \vee p_{|L|+2} \wedge p_{j-1}), & \text{if the } l\text{-th instruction of } L \text{ is } \texttt{JZ}_{2,k} \text{ and } k \le j; \\ p_l \wedge \bar{p}_{l+1} \wedge p_{|L|+2} \wedge p_{j-1}, & \text{if the } l\text{-th instruction of } L \text{ is } \texttt{JZ}_{2,k} \text{ and } k > j; \\ p_l \wedge \bar{p}_{l+1} \wedge p_j, & \text{if the } l\text{-th instruction of } L \text{ is } \texttt{Halt} \text{ and } l < |L|; \\ p_l \wedge p_j, & \text{if the } l\text{-th instruction of } L \text{ is } \texttt{Halt} \text{ and } l = |L|. \end{cases}$$

$f_{|L|+1} = \bigvee_{l=1}^{|L|} \varphi_{|L|+1,l}$ where $\varphi_{|L|+1,l}$ is defined as follows:

$$\varphi_{|L|+1,l} = \begin{cases} p_l \wedge \bar{p}_{l+1}, & \text{if the } l\text{-th instruction of } L \text{ is } \texttt{Inc}_1; \\ p_l \wedge \bar{p}_{l+1} \wedge p_{|L|+2}, & \text{if the } l\text{-th instruction of } L \text{ is } \texttt{Dec}_1; \\ p_l \wedge \bar{p}_{l+1} \wedge p_{|L|+1}, & \text{if the } l\text{-th instruction of } L \text{ is } \texttt{Inc}_2, \texttt{Dec}_2, \texttt{JZ}_{1,k} \text{ or } \texttt{JZ}_{2,k}; \\ p_l \wedge \bar{p}_{l+1} \wedge p_{|L|+1}, & \text{if the } l\text{-th instruction of } L \text{ is } \texttt{Halt} \text{ and } l < |L|; \\ p_l \wedge p_{|L|+1}, & \text{if the } l\text{-th instruction of } L \text{ is } \texttt{Halt} \text{ and } l = |L|. \end{cases}$$

$f_{|L|+3} = \bigvee_{l=1}^{|L|} \varphi_{|L|+3,l}$ where $\varphi_{|L|+3,l}$ is defined as follows:

[2] The original theorem in [**?**] is stated over natural numbers (non-negative integers). However, it can be extended to integers (including negative integers).

$$\varphi_{|L|+3,l} = \begin{cases} p_l \wedge \bar{p}_{l+1} \wedge p_{j-1}, & \text{if the } l\text{-th instruction of } L \text{ is } \texttt{Inc}_1; \\ p_l \wedge \bar{p}_{l+1} \wedge p_{j+1}, & \text{if the } l\text{-th instruction of } L \text{ is } \texttt{Dec}_1; \\ p_l \wedge \bar{p}_{l+1} \wedge p_j, & \text{if the } l\text{-th instruction of } L \text{ is } \texttt{Inc}_2, \texttt{Dec}_2, \texttt{JZ}_{1,k} \text{ or } \texttt{JZ}_{2,k}; \\ p_l \wedge \bar{p}_{l+1} \wedge p_j, & \text{if the } l\text{-th instruction of } L \text{ is } \texttt{Halt} \text{ and } l < |L|; \\ p_l \wedge p_j, & \text{if the } l\text{-th instruction of } L \text{ is } \texttt{Halt} \text{ and } l = |L|. \end{cases}$$

The definitions of $f_{|L|+2}$ and $f_{|L|+4}$ can be similarly defined.

▶ **Example 20.** Suppose that a 2-counter machine M has a list L: $1 : \texttt{Inc}_1$; $2 : \texttt{JZ}_{12,01}$; $3 : \texttt{Halt}$. and the initial configuration C_0 is with $C_0(r_0) = 1$ and $C_0(r_1) = C_0(r_2) = 0$. As illustrated in Figure 1, the ICG $\Pi^*(M, C_0)$ is played on a two-dimensional infinite grid.

The list L has three instructions. It follows that $n = |L| + 3 = 6$ and $m = 2^n + n + 2 = 72$. Row $y_i = i \cdot m + 1$ corresponds to the i-th step configuration For example, the initial configuration C_0 is encoded into row 1 while the successor configurations $M^1(C_0)$, $M^2(C_0)$, and $M^3(C_0)$ appear at rows $73(1\times72+1)$, $145(2\times72+1)$, and $217(3\times72+1)$, respectively. The first three cells $(1,1)$, $(2,1)$ and $(3,1)$ are used to represent $C_0(r_0)$ via unary encoding. Since $C_0(r_0) = 1 = (100)_1$, $w(1,1) = 1$ and $w(2,1) = w(3,1) = 0$. Similarly, cells $(4,1), (6,1)\cdots$ denotes $C_0(r_1)$ while cells $(5,1), (7,1)\cdots$ represents $C_0(r_2)$. Since $C_0(r_1) = C_0(r_2) = 0 = (000\cdots)_1$, $w(x,1) = 0$ for $x \geq 4$. In order to ensure that $w(1,1) = 1$, we use action *init* to link cell $(1,1)$ to $(0,1)$.

In the following, we elaborate the single-step update $M(C_0)$ of C_0. We use $|L| + 5 = 8$ Boolean functions f_j over $n = 6$ variables. We consider only column 2, which updated by function f_2. $f_2 = \varphi_{2,1} \vee \varphi_{2,2} \vee \varphi_{2,3}$ where each $\varphi_{2,l}$ corresponds to the l-th instruction of L. Since $1 : \texttt{Inc}_1 \in L$, $\varphi_{2,1} = p_1 \wedge \bar{p}_2 \wedge p_1 = p_1 \wedge \bar{p}_2$. Similarly, $\varphi_{2,2} = p_2 \wedge \bar{p}_3 \wedge (\bar{p}_4 \vee p_1)$ and $\varphi_{2,3} = p_3 \wedge p_2$. CombingCombining these three disjuncts yields $f_2 = (p_1 \wedge \bar{p}_2) \vee (p_2 \wedge \bar{p}_3 \wedge (\bar{p}_4 \vee p_1)) \vee (p_3 \wedge p_2)$.

By the definition of the offset $o(j,k)$, we get that $o(j,k) = k - j$ for function $j = 2$. So $o(2,1) = 1 - 2 = -1, \cdots, o(2,6) = 6 - 2 = 4$. In addition, the value of v_1 for column 2 is 2. Function f_2 for $w(2,73)$ takes $w(1,1), w(2,1), \cdots w(6,1)$ as input variables.

Each cell $(2, k+1)$ corresponds to the negation of the k-th variable $w(k,1)$ for $1 \leq k \leq 6$. Action $neglit_{2,k}$ transits cell $(2, k+1)$ to $(k,1)$, labeled by a red arrow. It follows that $w(2, k+1) = \neg w(k+1k, 1)$, for example, $w(2,2) = \neg w(1,1) = 0$ and $w(2,7) = \neg w(6,1) = 1$.

Each cell $(2, n+t+1)$ corresponds to the t-th minterm over $w(1,1), w(2,1), \cdots w(6,1)$ for $1 \leq t \leq 2^n$. According to the effect of $term_{2,k,t}$, each cell $(2, n+t+1)$ has multiple successor cells $(k,1)$ or $(2, k+1)$, labeled by a blue arrow. If $p_k \in \tau_t$, $term_{2,k,t}$ transits $(2, n+t+1)$ to $(2, k+1)$; otherwise, $(2, n+t+1)$ moves to $(k,1)$. For example, cell $(2,8)$, corresponding to $w(1,1) \wedge w(2,1) \wedge \cdots \wedge w(6,1)$ has 6 successor cells $(2,2), \cdots, (2,7)$. Cell $(2,71)$, corresponding to $\neg w(1,1) \wedge \neg w(2,1) \wedge \cdots \wedge \neg w(6,1)$ has 6 successor cells $(1,1), \cdots, (6,1)$.

Cells $(2,73)$ and $(2,72)$ corresponds to $f_2(w(1,1), w(2,1), \cdots w(6,1))$ and its negation, respectively. Action $ndnf_{2,t}$ moves cell $(2,72)$ to $(2, n+t+1)$ where $t \in F_{f_2}$ is the index of minterm appearing in the CDNF formula of f_2. Hence, $w(2,72) = \bigwedge_{t \in F_{f_2}} \neg w(2, n+t+1) = \neg f_2(w(1,1), w(2,1), \cdots, w(6,1)) = 0$. Action dnf transits cell $(2,73)$ to $(2,72)$. It follows that $w(2,73) = \neg w(2,72) = f_2(w(1,1), w(2,1), \cdots, w(6,1)) = 1$.

For the $M^2(C_0)$ and $M^3(C_0)$, we get that $w(2,145) = 0$ and $w(2,217) = 1$.

In addition, M does not halt on C_0. Hence, $w(0, 72i+1) = 0$ for $i \geq 1$.

All states of Mark game are encoded on cells on row 0. In Mark game, a player can take either 1 or $\lfloor \frac{x}{2} \rfloor$ chips. For example, the cell $(5,0)$, a player can transits to cell $(4,0)$ or $(2,0)$ via action *take*, marked by black arrows, respectively. In addition, every cell $(x,0)$ links to $(0, 72i+1)$ for $i \geq 1$. The integer x is a winning state in Mark game iff $w(x,0) = 1$. It is

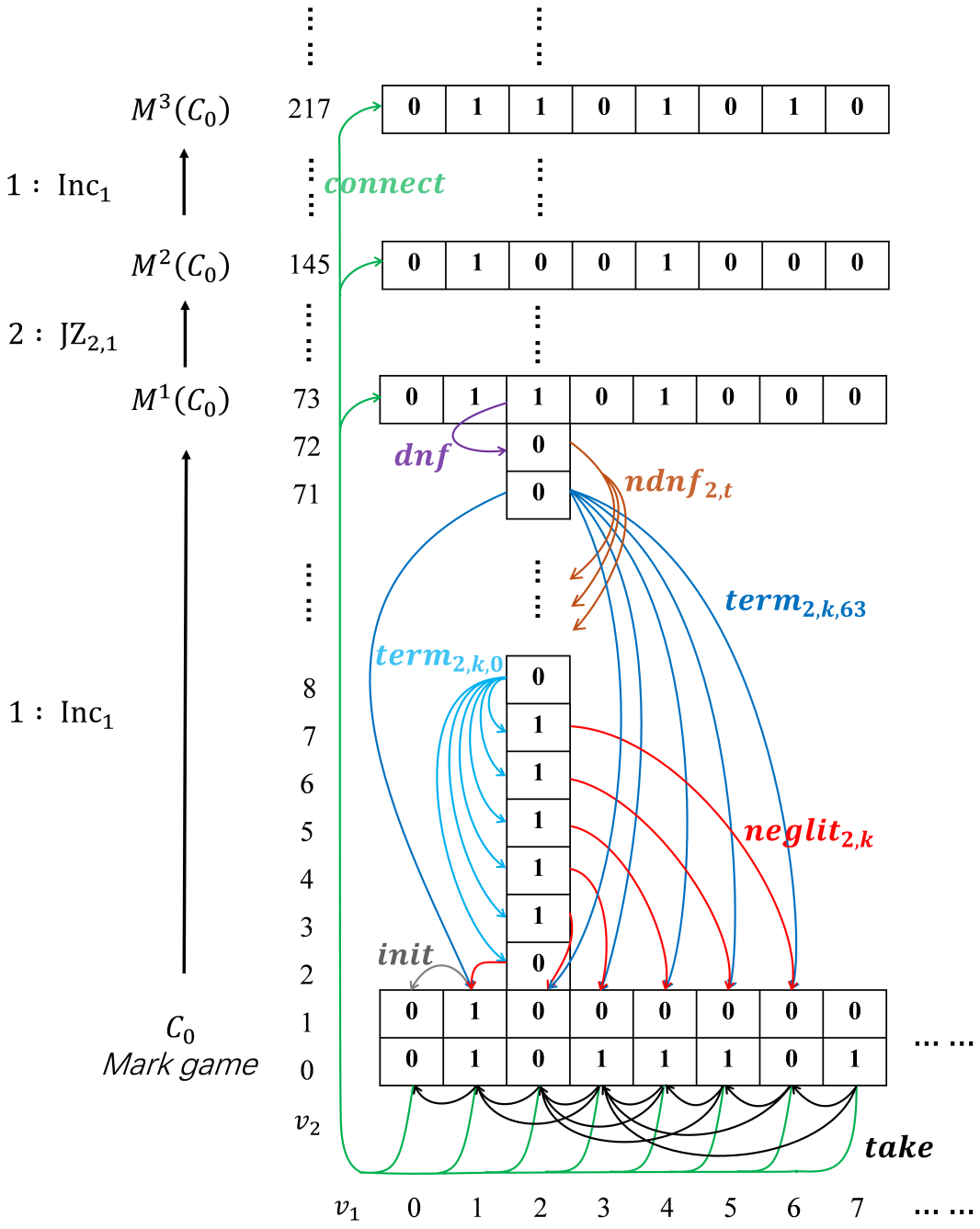


Figure 1 The grid of the ICG $\Pi^*(M, C_0)$.

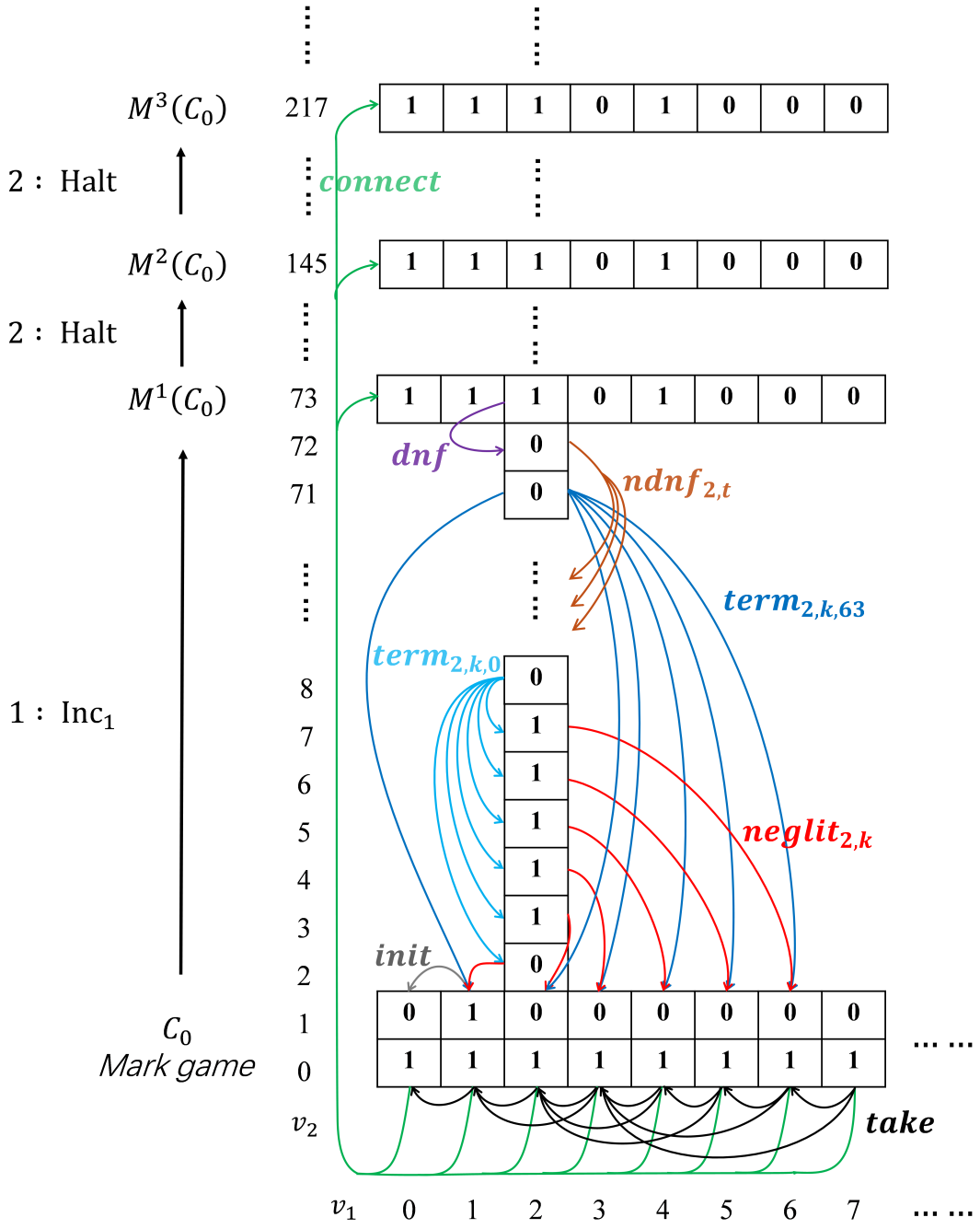


Figure 2 The grid of the ICG $\Pi^*(M', C_0)$.

easily verified that 0 and 2 are losing states in Mark game while 1 and 3 are winning states. So $w(0,0) = w(2,0) = 0$ and $w(1,0) = w(3,0) = 1$.

We consider another 2-counter machine M' with $L : 1 : \texttt{Inc}_1;\ 2 : \texttt{Halt}$. The ICG $\Pi^*(M', C_0)$ is shown in Figure 2. $M'^1(C_0)$ is a halting configuration, so are the successor ones $M'^2(C_0), M'^3(C_0), \cdots$. Hence, $w(0, 72i+1) = 1$ for $i \geq 1$ and $w(x, 0) = 1$ for $x \geq 0$. ◀

A state s of $\Pi^*(M, C_0)$ corresponds to cell $(v_1(s), v_2(s))$. For convenience, we use $w(s)$ to denote the Boolean variable $w(v_1(s), v_2(s))$.

▶ **Lemma 21.** *Let M be a 2-counter machine, C_0 the initial configuration, and $\Pi^*(M, C_0)$ the ICG defined in Definition 11. Let s be every legal state of $\Pi^*(M, C_0)$. Let $n = |L|+3$ and $m = 2^n+n+2$. Let* $i = \begin{cases} \lfloor \frac{v_2(s)}{m} \rfloor, \text{ if } v_2(s) \not\equiv_m 0; \\ \frac{v_2(s)}{m} - 1, \text{ otherwise.} \end{cases}$ *and* $j = \begin{cases} v_1(s), \text{ if } 0 \leq v_1(s) \leq |L|+2; \\ |L|+3, \text{ if } v_1(s) = |L|+3, |L|+5, \cdots; \\ |L|+4, \text{ if } v_1(s) = |L|+4, |L|+6, \cdots. \end{cases}$

Let $y_i = i \cdot m + 1$ and $y_{i-1} = (i-1) \cdot m + 1$.

Then,

- $w(s) = \neg w(v_1(s) + o(j,k)(s), y_i)$ *if* $s \models v_1 \geq 0 \wedge v_2 \equiv_m k+1$, $i \geq 0$, *and* $1 \leq k \leq n$;
- $w(s) = \tau_t(w(v_1(s) + o(j,1)(s), y_i), \cdots, w(v_1(s) + o(j,n)(s), y_i))$ *if* $s \models v_1 \geq 0 \wedge v_2 \equiv_m n+t+1$, $i \geq 0$ *and* $1 \leq t \leq 2^n$;
- $w(s) = \neg f_j(w(v_1(s) + o(j,1)(s), y_i), \cdots, w(v_1(s) + o(j,n)(s), y_i))$ *if* $s \models v_1 \geq 0 \wedge v_2 \geq 1 \wedge v_2 \equiv_m 0$ *and* $i \geq 0$;
- $w(s) = f_j(w(v_1(s) + o(j,1)(s), y_{i-1}), \cdots, w(v_1(s) + o(j,n)(s), y_{i-1}))$, *if* $s \models v_1 \geq 0 \wedge v_2 \geq m+1 \wedge v_2 \equiv_m 1$ *and* $i \geq 1$.

Proof. We prove by induction on i.

Base case ($i = 0$): We prove by the following situations:

- $s \models v_1 \geq 0 \wedge v_2 \equiv_m k+1$ and $1 \leq k \leq n$: There is only one action $neglit_{j,k}$ applicable in s, which yields state s' s.t. $v_1(s') = v_1(s) + o(j,k)(s)$ and $v_2(s') = v_2(s) - k = y_i$. Thus, s' is the only successor state of s s.t. $s' \models v_1 \geq 0 \wedge v_2 \geq m+1 \wedge v_2 \equiv_m 1$. It follows that $w(s) = \neg w(s') = \neg w(v_1(s) + o(j,k)(s), y_i)$.
- $s \models v_1 \geq 0 \wedge v_2 \equiv_m n+t+1$ and $1 \leq t \leq 2^n$: There are k actions $term_{j,k,t}$'s applicable in s for $1 \leq k \leq n$. Each action $term_{j,k,t}$ transits state s to state s'_k where

$$v_1(s'_k) = \begin{cases} v_1(s), & \text{if } p_k \in \tau_t; \\ v_1(s) + o(j,k)(s), & \text{if } \bar{p}_k \in \tau_t. \end{cases} \text{ and } v_2(s'_k) = \begin{cases} v_2(s) - n - t + o(j,k)(s), & \text{if } p_k \in \tau_t; \\ v_2(s) - n - t, & \text{if } \bar{p}_k \in \tau_t. \end{cases}$$

If $p_k \in \tau_t$, then $s'_k \models v_1 \geq 0 \wedge v_2 \equiv_m k+1$. In addition $i = 0$. By the above case, $w(s'_k) = \neg w(v_1(s'_k) + o(j,k)(s'_k), y_i)$. It can be verified that $v_1(s) + o(j,k)(s) = v_1(s'_k) + o(j,k)(s'_k)$. Hence $w(s'_k) = \neg w(v_1(s) + o(j,k)(s), y_i)$.

If $\bar{p}_k \in \tau_t$, then $s'_k \models v_1 \geq 0 \wedge v_2 = 1$. In addition $i = 0$. It can be verified that $v_1(s) + o(j,k)(s) = v_1(s'_k)$. Hence $w(s'_k) = w(v_1(s) + o(j,k)(s), y_i)$.

In summary, $w(s) = \bigwedge_{1 \leq k \leq n} \neg w(s'_k) = \left(\bigwedge_{p_k \in \tau_t} w(v_1(s) + o(j,k)(s), y_i)\right) \wedge \left(\bigwedge_{\neg p_k \in \tau_t} \neg w(v_1(s) + o(j,k)(s), y_i)\right) = \tau_t(w(v_1(s)+o(1,j)(s), y_i), \cdots, w(v_1(s)+o(n,jj,n)(s), y_i))$.

- $s \models v_1 \geq 0 \wedge v_2 \geq 1 \wedge v_2 \equiv_m 0$: There are t actions $ndnf_{j,t}$'s applicable in s for $t \in F_{f_j}$. Each action $ndnf_{j,t}$ transits state s to a state s'_t s.t. $v_1(s'_t) = v_1(s)$ and $v_2(s'_t) = v_s2(s) - 2^n + t$. By the above case, $w(s'_t) = \tau_t(w(v_1(s'_t) + o(j,1)(s'_t), y_i), \cdots, w(v_1(s'_t) + o(j,n)(s'_t), y_i))$. It follows that $s'_t \models v_1 \geq 0 \wedge v_2 \equiv_m n+t+1$. It can be verified that $v_1(s) + o(j,k)(s) = v_1(s'_t) + o(j,k)(s'_t)$ for $1 \leq k \leq n$. In summary, $w(s) = \bigwedge_{t \in F_{f_j}} \neg \tau_t(w(v_1(s) +$

$o(j,1)(s),y_i),\cdots,w(v_1(s)+o(j,n)(s),y_i)) = \neg f_j(w(v_1(s)+o(j,1)(s),y_i),\cdots,w(v_1(s)+o(j,n)(s),y_i))$.

Inductive step ($i>0$): We only prove by the following situation: $s \models v_1 \geq 0 \wedge v_2 \geq m+1 \wedge v_2 \equiv_m 1$ and $i \geq 1$. The other situations can be similarly proved in the base case. There is only one action dnf applicable in s, which yields in state s' s.t. $v_1(s') = v_1(s)$ and $v_2(s') = v_2(s)-1$. Thus, $w(s) = \neg w(s')$ and $s' \models v_1 \geq 0 \wedge v_2 \geq 1 \wedge v_2 \equiv_m 0$ and $i' \geq 0$ and $i' = \frac{v_2(s')}{m} - 1 = i-1$. By the inductive hypothesis, $w(s') = \neg f_j(w(v_1(s')+o(j,1)(s'),y_{i'}),\cdots,w(v_1(s')+o(j,n)(s'),y_{i'}))$. It can be verified that $v_1(s)+o(j,k)(s) = v_1(s')+o(j,k)(s')$ for $1 \leq k \leq n$. It follows that $w(s) = f_j(w(v_1(s)+o(j,1)(s),y_{i-1}),\cdots,w(v_1(s)+o(j,n)(s),y_{i-1}))$. ◀

Proof of Lemma 12

Proof. We prove by induction on i.

Base case ($i=0$): By the construction of $\Pi^*(M,C_0)$ (cf. Definition 11), we get the following

- $C_0(r_0) = w(1,1)\cdots w(|L|,1)$;
- $C_0(r_1) = w(|L|+1,1)w(|L|+3,1)\cdots$;
- $C_0(r_2) = w(|L|+2,1)w(|L|+4,1)\cdots$.

Inductive step ($i \geq 1$): Let $y_{i-1} = (i-1)\cdot m+1$. By inductive step, $M^{i-1}(C_0)(r_0) = w(1,y_{i-1})\cdots w(|L|,y_{i-1})$, $M^{i-1}(C_0)(r_1) = w(|L|+1,y_{i-1})w(|L|+3,y_{i-1})\cdots$, and $M^{i-1}(C_0)(r_2) = w(|L|+2,y_{i-1})w(|L|+4,y_{i-1})\cdots$. Let $l' = M^{i-1}(C_0)(r_0)$. We only verify the situation where $l' : \texttt{Inc}_1 \in L$. The other situations can be similarly verified.

We first prove that $M^i(C_0)(r_0) = w(1,y_i)\cdots w(|L|,y_i)$. Since $M^{i-1}(C_0)(r_0) = w(1,y_{i-1})\cdots w(|L|,y_{i-1})$, we get that $w(x,y_{i-1}) = 1$ for $1 \leq x \leq l'$ and $w(x,y_{i-1}) = 0$ for $l'+1 \leq x \leq |L|$. By Lemma 21, $w(1,y_i) = f_1(w(1,y_{i-1}),\cdots,w(|L|+3,y_{i-1})) = 1$ and $w(x,y_i) = f_x(w(1,y_{i-1}),\cdots,w(|L|+3,y_{i-1})) = \bigvee_l \varphi_{x,l} = w(l',y_{i-1}) \wedge \neg w(l'+1,y_{i-1}) \wedge w(x,y_{i-1})$ for $1 \leq x \leq |L|$. So $w(x,y_i) = 1$ for $1 \leq x \leq l'+1$ and $w(x,y_i) = 0$ for $l'+2 \leq x \leq |L|$. Clearly, $M^i(C_0)(r_0) = M^{i-1}(C_0)(r_0)+1$ and $M^i(C_0)(r_0) = w(1,y_i)\cdots w(|L|,y_i)$.

We then prove that $M^i(C_0)(r_1) = w(|L|+1,y_i)w(|L|+3,y_i)\cdots$. Let $z = M^{i-1}(C_0)(r_1)$. Since $M^{i-1}(C_0)(r_1) = w(|L|+1,y_{i-1})\cdots w(|L|+3,y_{i-1})$, we get that $w(|L|+2\cdot j-1,y_{i-1}) = 1$ for $1 \leq j \leq z$ and $w(|L|+2\cdot j-1,y_{i-1}) = 0$ for $j > z$. By Lemma 21, $w(|L|+1,y_i) = f_{|L|+1}(w(1,y_{i-1}),\cdots,w(|L|+3,y_{i-1})) = \bigvee_l \varphi_{x,l} = w(l',y_{i-1}) \wedge \neg w(l'+1,y_{i-1}) = 1$, $w(|L|+2\cdot j-1,y_i) = f_{|L|+3}(w(1,y_{i-1}),\cdots,w(|L|,y_{i-1}),w(|L|+2\cdot j-3,y_{i-1}),w(|L|+2\cdot j-1,y_{i-1}),w(|L|+2\cdot j+1,y_{i-1})) = 1$ for $2 \leq j \leq l'+1$, and $w(|L|+2\cdot j-1,y_i) = 0$ for $j > l'+1$. Clearly, $M^i(C_0)(r_1) = M^{i-1}(C_0)(r_1)+1$ and $M^i(C_0)(r_1) = w(|L|+1,y_i)w(|L|+3,y_i)\cdots$.

Similarly to the above proof, we can prove that $M^i(C_0)(r_2) = w(|L|+2,y_i)w(|L|+4,y_i)\cdots$.

Finally, we prove that $M^i(C_0)$ is halting iff $w(0,y_i) = 1$ for $i \geq 1$. $w(0,y_i) = f_0 = \bigvee_{l:\texttt{Halt}\in L \text{ and } 1\leq l<|L|}(f_l \wedge \neg f_{l+1}) \vee f_{|L|}$. The current instruction is `Halt` iff $w(0,y_i) = 1$.

For $x = |L|+1$, $w(x,y_i)$ can be defined by a Boolean function $f_{|L|+1}$ over $w(1,y_{i-1}),\cdots,w(|L|,y_{i-1}),w(|L|+1,y_{i-1})$ and $w(|L|+3,y_{i-1})$. For $x = |L|+3,|L|+5,\cdots$, the value of each $w(x,y_i)$ is determined by a Boolean function $f_{|L|+3}$ over $w(1,y_{i-1}),\cdots,w(|L|,y_{i-1}),w(x-2,y_{i-1}),w(x,y_{i-1})$ and $w(x+2,y_{i-1})$. ◀

B Proofs of Lemmas 15 and 18

Proof of Lemma 15

Proof. We first prove the termination of Algorithm 1. Since any terminating ICG always ends in a finite number of moves, s must satisfy some ψ or φ (lines 2, 6 & 8). Hence, the loop (lines 4 - 8) is guaranteed to terminate. Algorithm 1 always terminates.

We now prove the soundness of Algorithm 1 by contraposition. Suppose that s is not a winning state. It follows that s is a losing state. It will satisfy some formula ψ iteratively constructed in lines 1 & 7. Algorithm 1 terminates with $\bot$ (lines 2 & 8).

We finally prove the completeness of Algorithm 1. Suppose that s is a winning state. It will satisfy some formula φ iteratively constructed in line 5. Algorithm 1 terminates with $\bot$ (line 6). ◀

Proof of Lemma 18

Proof. ($\Rightarrow$): Assume that ϕ is the winning formula. It follows that every state s satisfying $\mathcal{C} \wedge \phi$ is winning and every state s satisfying $\mathcal{C} \wedge \neg\phi$ is losing. We prove the three constraints illustrated in Definition 17 are valid.

1. $\mathcal{C} \wedge \mathcal{E} \rightarrow \neg\phi$: Let s be a state satisfying $\mathcal{C} \wedge \mathcal{E}$. It follows that s is a legal terminal state. By the first item of the definition of winning and losing states (cf. Definition 14), s is a losing state. Hence, $s \models \neg\phi$ and the first constraint is valid.
2. $(\mathcal{C} \wedge \phi) \rightarrow \exists \mathcal{V}'[\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \neg\phi)[\mathcal{V}/\mathcal{V}']]$: Let s be a state satisfying $\mathcal{C} \wedge \phi$. It follows that s is a legal winning state. By the second item of the definition of winning and losing states, there is at least one applicable action a in s s.t. $\tau(s, a)$ is a losing state . Hence, $s \models \exists \mathcal{V}'[\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \neg\phi)[\mathcal{V}/\mathcal{V}']]$ and the second constraint is valid.
3. $(\mathcal{C} \wedge \neg\phi) \rightarrow \forall \mathcal{V}'[\mathcal{T}(\mathcal{A}) \rightarrow (\mathcal{C} \wedge \phi)[\mathcal{V}/\mathcal{V}']]$: The proof of the validity of the third constraint is the same as that of the second constraint by using the third item of the definition of winning and losing states.

($\Leftarrow$): Assume that ϕ is a formula s.t. all of the constraints illustrated in Definition 17 is valid. We prove that every state s satisfying $\mathcal{C} \wedge \phi$ is winning and every state s satisfying $\mathcal{C} \wedge \neg\phi$ is losing by induction on s.

Base case (s is a legal terminal state): It follows that $s \models \mathcal{C} \wedge \mathcal{E}$. By the first constraint $\mathcal{C} \wedge \mathcal{E} \rightarrow \neg\phi$, we get that $s \models \neg\phi$. By the definition of winning formula (cf. Definition 3), s is a losing state.

Inductive step (s is a legal but not terminal state): Suppose that $s \models \phi$. By the second constraint $(\mathcal{C} \wedge \phi) \rightarrow \exists \mathcal{V}'[\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \neg\phi)[\mathcal{V}/\mathcal{V}']]$, there is a successor state $\tau(s, a)$ of s s.t. $\tau(s, a) \models \mathcal{C} \wedge \neg\phi$. By the inductive assumption, $\tau(s, a)$ is a losing state. By the second item of the definition of winning and losing states, s is a winning state. Similarly, we can prove that s is a losing state when $s \models \neg\phi$ by using the third constraint $(\mathcal{C} \wedge \neg\phi) \rightarrow \forall \mathcal{V}'[\mathcal{T}(\mathcal{A}) \rightarrow (\mathcal{C} \wedge \phi)[\mathcal{V}/\mathcal{V}']]$ and the third item of the definition of winning and losing states. ◀

C Extensions to Losing and Draw States and Formulas

In this section, we extend Theorems 9, 10, 13, 16, and 19 to losing (and/or draw) states and formulas. The following are decisions problems we investigate in this section.

1. The losing state problem (deciding if a state is losing for an LIA-definable ICG);
2. The draw state problem (deciding if a state is draw for an LIA-definable ICG);
3. The losing formula problem (deciding if an LIA formula exactly characterizes the set of losing states of an LIA-definable ICG);
4. The draw formula problem (deciding if an LIA formula exactly characterizes the set of losing states of an LIA-definable ICG);

5. The LIA-definability problem of losing formulas (deciding if there is an LIA formula that exactly characterizes the set of losing states of an LIA-definable ICG).

▶ **Theorem 22.** *The losing state problem for LIA-definable ICGs is undecidable.*

Proof. Let s be a state of $\Pi'(M)$ and C the corresponding configuration in M s.t. $v(s) = 3^{C(r_0)}5^{C(r_1)}7^{C(r_2)}b$. The state s is losing iff M is terminating in C. Since the halting problem of 2-counter machines is undecidable, deciding if a state is losing for an LIA-definable ICG is undecidable. ◀

▶ **Theorem 23.** *The draw state problem for LIA-definable ICGs is undecidable.*

Proof. It can be verified that any state is draw iff M is non-terminating in C. Therefore, deciding if a state is draw for an LIA-definable ICG is also undecidable. ◀

▶ **Theorem 24.** *The losing formula problem for LIA-definable ICGs is undecidable.*

Proof. Let ϕ be the formula $v \geq 0 \wedge v \not\equiv_4 0 \wedge v \not\equiv_4 2 \wedge \left(\bigvee_{1\leq l\leq |L|} \phi_{3,l,|L|}\right)$. The formulas ϕ exactly characterize the set of positive integers $3^l b$ where $1 \leq l \leq |L|$ and b is not a multiple of 2 or 3. It is easily verified that ϕ is the winning formula for $\Pi'(M)$ iff M is terminating on all configurations. Since the mortality problem of 2-counter machines is undecidable, deciding if an LIA-formula is the losing formula for an LIA-definable ICG is undecidable.

The formula ϕ exactly characterizes the set of positive integers $2^1 3^l b$ where $1 \leq l \leq |L|$ and b is not a multiple of 2 or 3. It can be verified that ϕ is the winning formula for $\Pi'(M)$ iff M is terminating on all configurations. Since the mortality problem of 2-counter machines is undecidable, deciding if an LIA-formula is a winning formula for an LIA-definable ICG is undecidable. ◀

▶ **Theorem 25.** *The draw formula problem for LIA-definable ICGs is undecidable.*

Proof. Let ϕ be the formula $\bot$. The formula ϕ denotes the empty set of integers. It can be verified that ϕ is the draw formula for $\Pi'(M)$ iff M is terminating on all configurations. Since the mortality problem of 2-counter machines is undecidable, deciding if an LIA-formula is the draw formula for an LIA-definable ICG is undecidable. ◀

▶ **Theorem 26.** *The LIA-definability problem of losing formulas for LIA-definable ICGs is undecidable.*

Proof. The ICG $\Pi^*(M, C_0)$ is terminating By Definition 14, we obtain that every legal state in $\Pi^*(M, C_0)$ is either winning or losing. Hence, the negation of the winning formula is the losing formula, and vice versa. The winning formula is LIA-definable for $\Pi^*(M, C_0)$ iff the losing formula also is. By Theorem 13, deciding if the set of losing states of $\Pi^*(M, C_0)$ is LIA-definable is undecidable ◀

▶ **Theorem 27.** *The losing state problem for LIA-definable terminating ICGs is decidable.*

Proof. Algorithm `IsWinningState`(Π, s) returns $\bot$ iff s is a losing state. Hence, Algorithm 1 is also complete and sound for losing states. It follows that deciding a state is winning for an LIA-definable terminating ICG is decidable. ◀

▶ **Theorem 28.** *The draw state problem for LIA-definable terminating ICGs is decidable.*

Proof. Since any terminating ICG has no draw state, any state is not draw. ◀

Algorithm 2 `IsWinningState'`(Π, s)

Input: $\Pi = \langle \mathcal{V}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$: an LIA-definable ICG;
s: a legal state.
Output: $\top$, if s is a winning state under the misère play rule; $\bot$, otherwise.

1 $\psi \leftarrow \bot$ and $\varphi \leftarrow (\mathcal{C} \wedge \mathcal{E})$
2 **if** $s \models \varphi$ **then return** $\top$;
3 **else**
4 **while true do**
5 $\psi' \leftarrow$ a quantifier-free formula equivalent to $\forall \mathcal{V}' \{\mathcal{T}(\mathcal{A}) \rightarrow (\mathcal{C} \wedge \varphi)[\mathcal{V}/\mathcal{V}']\}$
6 $\psi \leftarrow \psi \vee \psi'$
7 **if** $s \models \mathcal{C} \wedge \psi$ **then return** $\bot$;
8 $\varphi \leftarrow$ a quantifier-free formula equivalent to $\exists \mathcal{V}' \{\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \psi)[\mathcal{V}/\mathcal{V}']\}$
9 $\varphi \leftarrow \varphi \vee \varphi'$
10 **if** $s \models \mathcal{C} \wedge \varphi$ **then return** $\top$;

▶ **Theorem 29.** *The losing formula problem for LIA-definable terminating ICGs is decidable.*

Proof. As said in the proof of Theorem 26, the negation of the winning formula is the losing formula, and vice versa. We reduce the losing formula problem to the winning formula problem for LIA-definable terminating ICGs. By Theorem 19, the latter decision problem is decidable. So is the former decision problem. ◀

▶ **Theorem 30.** *The draw formula problem for LIA-definable terminating ICGs is decidable.*

Proof. Since terminating ICGs have no draw state, the draw formula is $\bot$. It follows that an LIA formula ϕ is the draw formula iff $\neg\phi$ is valid. Hence, deciding if an LIA-formula is a draw formula for an LIA-definable terminating ICG is decidable. ◀

D Extensions to the Misère Play Rule

The terminating and cyclic properties of ICGs does not depend on the play rule, so the terminating and cyclic problems of LIA-definable ICGs are undecidable under the misère play rule.

For the ICGs $\Pi'(M)$ and $\Pi^*(M, C_0)$ defined in Definition 8, we have

1. A state is losing under the normal play rule iff it is winning under the misère play rule;
2. A state is winning under the normal play rule iff it is losing under the misère play rule;
3. A state is draw under the normal play rule iff it is draw under the misère play rule.

Under the misère play rule, in the proof of Theorems 9 and 22, a state with value $2^1 3^{C(r_0)} 5^{C(r_1)} 7^{C(r_2)} b$ is a losing state while a state with value $3^{C(r_0)} 5^{C(r_1)} 7^{C(r_2)} b$ is a winning state. Similarly, in the proof of Theorems 10 and 24, ϕ is $v \geq 0 \wedge v \not\equiv_4 0 \wedge v \not\equiv_4 2 \wedge \left(\bigvee_{1 \leq l \leq |L|} \varphi_{3,l,|L|}\right)$ and $v \geq 0 \wedge v \equiv_4 2 \wedge \left(\bigvee_{1 \leq l \leq |L|} \varphi_{3,l,|L|}\right)$, respectively. In addition, M halts in C_0 iff the set of losing states in $\Pi^*(M, C_0)$ is LIA-definable. In summary, the six theorems (Theorems 9, 10, 13, 22, 24 and 26) also hold under the misère play rule.

For the winning and losing state problems for terminating LIA-definable ICGs, we can modify Algorithm 1 as Algorithm 2. In Algorithm 2, ψ and φ are initialized as $\bot$ and $\mathcal{C} \wedge \mathcal{E}$, respectively. In the loop (lines 4 - 10), it first construct the formulas ψ' and ψ and followed by φ' and φ. It can be verified that Algorithm 2 always terminates, and is sound and complete for the winning state problem under the misère play rule.

▶ **Definition 31.** Let $\Pi = \langle \mathcal{V}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be a terminating ICG. The constraints for the winning formula ϕ of Π under the misère play rule are as follows:

1. $\mathcal{C} \wedge \mathcal{E} \rightarrow \phi$;
2. $(\mathcal{C} \wedge \neg\mathcal{E} \wedge \phi) \rightarrow \exists \mathcal{V}'[\mathcal{T}(\mathcal{A}) \wedge \neg\phi[\mathcal{V}/\mathcal{V}']]$;
3. $(\mathcal{C} \wedge \neg\phi) \rightarrow \forall \mathcal{V}'[\mathcal{T}(\mathcal{A}) \rightarrow (\mathcal{C} \wedge \phi)[\mathcal{V}/\mathcal{V}']]$.

For the winning and losing formula problems for terminating LIA-definable ICGs, we can modify Definition 17 via replacing the first and second constraints by $\mathcal{C} \wedge \mathcal{E} \rightarrow \phi$ and $(\mathcal{C} \wedge \neg\mathcal{E} \wedge \phi) \rightarrow \exists \mathcal{V}'[\mathcal{T}(\mathcal{A}) \wedge \neg\phi[\mathcal{V}/\mathcal{V}']]$, respectively, illustrated in Definition 31. The first constraint means that any terminal state is a winning state. The second constraint says that every non-terminal state such that some of its successor states are losing states is a winning state. It can be verified that a formula ϕ is the winning formula of Π under the misère play rule iff all of the constraints illustrated in Definition 31 are valid.